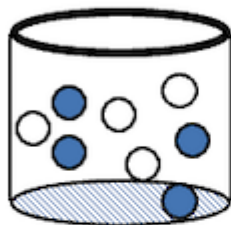


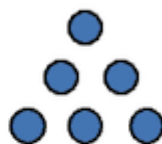
Proof Systems and Static Analysis

An Example

Goal: implement this game

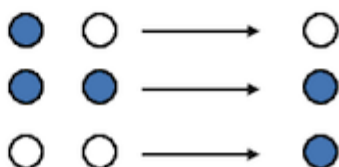


jar of balls



stock of black
balls

Rules



@NéstorCataño

@NéstorCataño

The Problem

Question 1: Does the game stop?

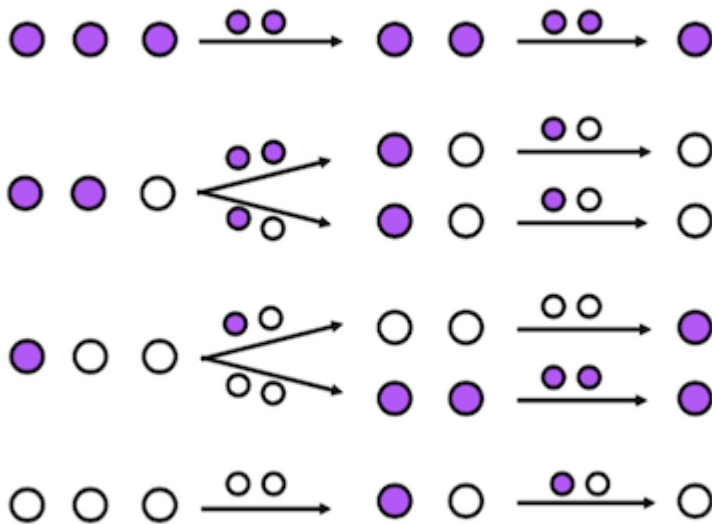
Question 2: If so, can you say anything about the **color** of the last ball by knowing the original configuration?

Why are these questions important?

@NéstorCataño

@NéstorCataño

Testing (# balls = 3)



@NéstorCataño

A Formal Model

b = black **w** = white **f** = transition function

$$f(b, w) = \begin{matrix} f(b - 2 + 1, w) \\ f(b - 1, w - 1 + 1) \\ f(b + 1, w - 2) \end{matrix} \begin{matrix} = f(b - 1, w) \\ = f(b - 1, w) \\ = f(b + 1, w - 2) \end{matrix}$$

$$f(0, 1) = (0, 1)$$

$$f(1, 0) = (1, 0)$$

@NéstorCataño

@NéstorCataño

Example : Reasoning

Theorem 1: The game “stops.”

Theorem 2:

$f^*(b, w) = \text{if odd}(w)$
 then $(0, 1)$
 else $(1, 0)$.

How might you prove this?

@NéstorCataño

@NéstorCataño

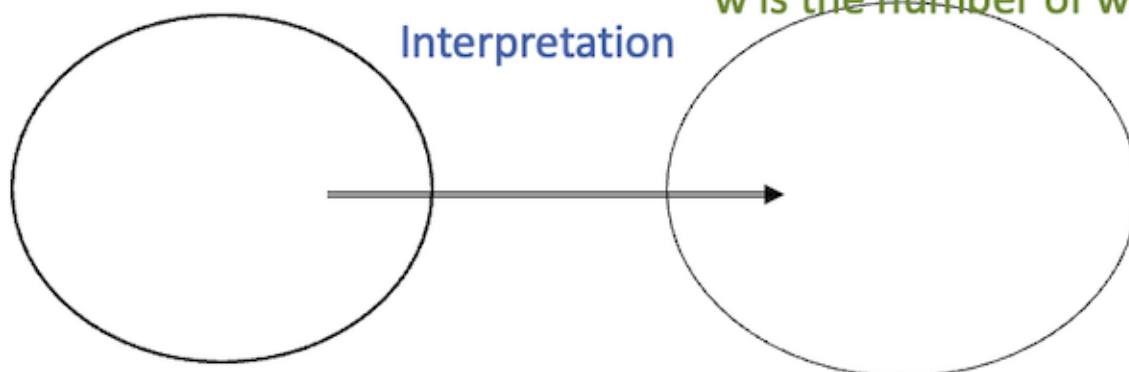
Basic Elements

Formal System

Syntax (formal language)
 b, w, f

Semantics (meaning)
 b is the number blacks
 w is the number of whites

Interpretation



Inference system
 (rules of reasoning)

Validity

$(b - 2 + 1, w) = (b - 1, w)$

if b is even then $b-2$ is even

@NéstorCataño

@NéstorCataño

Principia Mathematica

- **Bertrand Russel:** (Principia Mathematica)
 - It is about the difficulty of telling “**right from wrong**”
- **David Hilbert:** any formal system should be
 1. **Consistent:** you would ever get a contradiction
 2. **Complete:** all truths/theorems are provable
 3. **Decidable:** an algorithm to decide if an assertion is true exists
- **Kurt Gödel:**
 - No mathematical system can be both **consistent** and **complete**

@NéstorCataño

@NéstorCataño



Consequences of Gödel's Incompleteness Theorem

- **Terminology (static analyzers)**
 - **False negative:** a vulnerability exists in the code, but the tool fails to detect it
 - **False positive:** the tool reports a vulnerability that in fact does not exist
- A static analyzer is **sound** if prevents **false negatives**, that is, it detects all real vulnerabilities
- A system is **complete** if prevents **false positives**, that is, only reports real vulnerabilities

Consequences of Gödel's Incompleteness Theorem

- You cannot build a compiler that detect all the errors of the program
 - Typing is undecidable in general
 - Sound procedures over-approximations

Gödel's incompleteness theorem shows that in any sufficiently expressive, consistent, and effectively axiomatizable formal system, there will always be true statements that cannot be proven within the system.

This has a deep analogy in the context of static analyzers (e.g. for detecting vulnerabilities) and compilers: just as no formal system can be both complete and consistent when reasoning about arithmetic, no static analyzer can be both sound (catching all real bugs) and complete (raising no false alarms) when reasoning about programs of realistic complexity.

To ensure soundness, static analyzers typically over-approximate program behaviour—conservatively assuming that certain program paths or values might lead to errors even if they don't in practice. This strategy guarantees that all real vulnerabilities are caught, but it also results in false positives, or false alarms, where the tool warns about code that is actually correct.

Note: By over-approximation, in order to enforce soundness, static analyzers often compromise completeness.

Consequently, static analysis tools must trade off between missing real errors (false negatives) and over-warning (false positives). Most practical tools prioritise soundness—ensuring that if they say nothing, the code is likely safe—at the cost of producing warnings for code that is actually correct. This unavoidable limitation echoes Gödel's insight: certain truths (or bugs) will always remain beyond the reach of mechanical reasoning.